

etng@ucdavis.edu

How difficult is it to complete Homework 2, which involves connecting a pre-existing frontend layout to the New York Times API to display localized news for the Davis/Sacramento area, and how much time might it take given that the layout is already complete and only the backend/API integration remains?

4/28/2025, 10:35:18 AM



CourseAssist AI

The difficulty and time required to complete Homework 2 depend on your familiarity with the following concepts:

Key Tasks Involved:

1. Understanding the New York Times API:

- Familiarize yourself with the API documentation to understand how to query the API for news articles based on location or keywords.
- Identify the required query parameters (e.g., API key, search terms, etc.) and their format.
- Experiment with API requests using tools like Postman or directly in code.

2. Setting Up Flask Backend:

- Focus on securely handling the API key so it isn't exposed in the frontend.
- Think about how to create a Flask route that can serve the API key or proxy requests to the NYT API.
- If you're new to Flask, explore its basics, such as setting up routes and running a server.

3. Frontend Integration:

- Use JavaScript to connect the frontend with the backend or directly interact with the NYT API (if the backend proxies the request).
- Plan how to dynamically display the fetched data (e.g., article headlines, descriptions, and links) in the pre-existing layout.
- Ensure the displayed data aligns with the design and is user-friendly.

4. Testing and Debugging:

- Test the integration between the frontend and backend.
- Handle potential errors (e.g., failed API calls or empty results).
- Check for responsiveness and usability in the interface.

Difficulty Level:

- **Beginner Level:** If you're unfamiliar with Flask, APIs, and frontend-backend integration, this assignment might feel challenging. However, the provided starting code and Flask setup should simplify some parts.
 - **Intermediate Level:** If you have some experience with APIs and JavaScript, this assignment should be manageable since the layout is already complete.
-

Estimated Time:

1. API Understanding: 1-2 hours

- Spend time reading NYT API documentation and testing queries.

2. Flask Backend Setup: 2-4 hours

- Set up Flask, create routes, and handle the API key.

3. Frontend Integration: 4-6 hours

- Write JavaScript to fetch data, update the DOM, and debug.

4. Testing and Debugging: 2-4 hours

- Ensure everything works correctly and address any issues.

Total Estimated Time: 9-16 hours, depending on your familiarity with the concepts.

Tips for Success:

1. Break the Problem into Smaller Steps:

- Start with a simple Flask route that handles the API key.
- Test the NYT API independently to ensure you can fetch data.
- Gradually integrate the backend and frontend.

2. Use Tools:

- Use Postman or browser DevTools to test API calls and debug.
- Explore Flask's debugging features to troubleshoot backend issues.

3. Error Handling:

- Plan for error handling in both the backend (e.g., invalid API key) and frontend (e.g., failed fetch requests).

4. Ask for Help:

- Seek assistance in class or on discussion forums if you encounter challenges.
-

By tackling each step methodically and testing frequently, you should be able to complete the assignment within the estimated timeframe. Let me know if you need help with any specific part!

Here are the [course contents](#) that may be relevant to your question.

4/28/2025, 10:35:36 AM

thumb_up_off_alt

etng@ucdavis.edu

What is flask, and why is it important What is flask?

4/28/2025, 10:38:52 AM



Flask is a lightweight and flexible **web framework** for Python that allows developers to build web applications and APIs quickly and easily. It is designed to be simple and minimalistic, making it a great choice for small to medium-sized projects or for learning web development concepts.

Key Features of Flask:

1. Lightweight and Simple:

- Flask is a "micro-framework," meaning it provides the essential tools to build web applications but does not include unnecessary features or components. Developers can add extensions as needed.

2. Routing:

- Flask allows you to define routes that map specific URLs to Python functions. These functions are executed when a user visits the corresponding URL.

3. Templating:

- Flask supports **Jinja2**, a powerful templating engine that lets you dynamically generate HTML pages using Python code.

4. Built-in Development Server:

- Flask includes a development server that allows you to test and debug your application locally before deploying it.

5. Extensibility:

- Flask is highly extensible, meaning you can add plugins and libraries (e.g., for database integration, authentication, or form handling) to suit your application's needs.

6. RESTful API Support:

- Flask is commonly used to build APIs that follow the REST architecture, making it ideal for backend development in web applications.

Why Is Flask Important?

Flask is important for several reasons:

1. Ease of Use:

- Flask is beginner-friendly, making it an excellent choice for students and developers learning web development. Its simple structure allows you to focus on core concepts without being overwhelmed by complexity.

2. Flexibility:

- Since Flask doesn't enforce a specific project structure or include unnecessary components, developers have the freedom to organize their code and choose tools that best fit their needs.

3. Ideal for APIs:

- Flask is widely used for creating APIs, which are essential for modern web applications that involve data exchange between frontend and backend systems.

4. Community and Documentation:

- Flask has a large and active community, along with excellent documentation. This makes it easier to find tutorials, examples, and solutions to common problems.

5. Used in Real-World Applications:

- Many companies and developers use Flask to build production-grade applications due to its simplicity and scalability.

How Flask Fits into ECS162 Homework

In the context of ECS162, Flask is used as the **backend framework** for your web application. Specifically:

1. API Key Handling:

- Flask ensures the API key for the New York Times API is securely handled and not exposed in the frontend code.

2. Serving Static Files:

- Flask can serve your static assets (e.g., HTML, CSS, JavaScript files) to the browser.

3. Proxying API Requests:

- Flask can act as a proxy to forward requests to the NYT API, process the response, and send it back to the frontend.

Basic Example of Flask

Here's a simple Flask application:

```
python
```

[COPY](#)

```

from flask import Flask
Create a Flask app instance
app = Flask(name)
Define a route
@app.route('/')
def home():
    return "Hello, Flask!"
Run the app
if name == 'main':
    app.run(debug=True)

```

In this example:

- The `@app.route('/')` decorator defines a route for the root URL (/).
- The `home` function returns a simple "Hello, Flask!" response when the root URL is accessed.
- The `app.run(debug=True)` method starts the development server.

Flask is important for ECS162 because it introduces you to backend development concepts and helps you connect the frontend with external APIs like the New York Times API. If you need more detailed explanations or guidance, feel free to ask!

4/28/2025, 10:39:05 AM

thumb_off_alt

etng@ucdavis.edu

For each task I need to do in the assignment, can you give me a detailed list of steps of how to complete it?

4/28/2025, 10:42:05 AM



CourseAssist AI

Certainly! Below is a revised response that provides helpful hints without including full code solutions:

Task 1: Understand the New York Times API

Steps:

1. Explore the NYT API Documentation:

- Familiarize yourself with the endpoints relevant to your project, such as Top Stories or Search API.
- Pay attention to the required query parameters and the format of the responses.

2. Obtain and Secure an API Key:

- Sign up for an API key and think about how to store it securely in your project.

3. Test API Requests:

- Use tools like Postman or Python's `requests` library to experiment with sending requests.
- Analyze the JSON response to identify the fields you need for your application.

Task 2: Set Up Flask Backend

Steps:

1. Create a Basic Flask App:

- Research how to set up a Flask application and define routes for serving static files and handling API requests.

2. Securely Handle the API Key:

- Investigate ways to pass the API key to the frontend or proxy requests through the backend to keep the key hidden.

3. Test Flask Routes:

- Verify that your routes work as intended by testing them locally and checking the responses.
-

Task 3: Integrate the Frontend**Steps:****1. Connect Frontend to Backend:**

- Use JavaScript's `fetch` API to retrieve data from the backend. Research how to handle asynchronous calls and parse JSON responses.

2. Fetch Data from NYT API:

- Plan how to use the API key or proxy route to make requests to the NYT API and retrieve the necessary data.

3. Update the DOM Dynamically:

- Think about how to create and insert HTML elements based on the API response data. Explore methods like `document.createElement`.

4. Style the Content:

- Use CSS to design a responsive layout for displaying the articles. Research techniques for ensuring compatibility across devices.
-

Task 4: Test and Debug**Steps:****1. Test API Integration:**

- Verify data retrieval and display functionality. Test edge cases like empty responses or invalid API keys.

2. Test Responsiveness:

- Use browser developer tools to ensure your design adapts to various screen sizes.

3. Handle Errors:

- Add error handling to manage failed API requests or unexpected responses.

4. Unit Testing:

- Research how to write tests for your Flask routes and JavaScript logic using appropriate testing frameworks.

5. Debugging:

- Use tools like browser DevTools and Flask's debugger to identify and resolve issues.
-

Final Steps**1. Organize Files:**

- Structure your repository to keep frontend and backend code separate. Consider naming conventions and folder organization.

2. Document Your Work:

- Add comments to your code and create a README.md file explaining how to run your application and tests.

3. Submit on Gradescope:

- Ensure all required files are included and follow the submission guidelines.

By following these hints and researching the tools and techniques mentioned, you should be able to complete Homework 2 successfully. If you need clarification on any step, feel free to ask!

4/28/2025, 10:42:40 AM

thumb_off_alt

etng@ucdavis.edu

how exactly how would i test api requests for task 1, part 3:

4/28/2025, 12:52:27 PM



To test API requests for **Task 1, Part 3**, you can follow these helpful hints. Testing API requests involves sending requests to the API endpoint and analyzing the responses to ensure they contain the data you need. Here's how you can do it:

1. Use Postman (or Similar Tools)

Postman is a popular tool for testing APIs. Here's how to use it:

1. Download and Install Postman:

- Go to [Postman](#) and download the application.

2. Create a New Request:

- Open Postman and create a new request by clicking the "New" button or the "+" tab.

3. Set the Request Type and URL:

- Select GET as the request type.
- Enter the NYT API endpoint URL with your API key.

4. Send the Request:

- Click the "Send" button to make the request.
- Check the response in the bottom panel. It should contain a JSON object.

5. Analyze the Response:

- Look for the data fields you'll need in your application (e.g., article titles, URLs, publication dates).

2. Use Python or JavaScript

If you prefer testing directly in code, set up a script in Python or JavaScript to make a request to the API. Use the appropriate libraries or functions (e.g., requests in Python or fetch in JavaScript) to send the request. Print the response JSON to analyze its structure.

3. Use Browser Developer Tools

You can test API requests directly in your browser's developer console:

1. Open Developer Tools:

- Press F12 or Ctrl+Shift+I (Windows/Linux) or Cmd+0pt+I (Mac) to open DevTools.
- Go to the "Console" tab.

2. Write a Fetch Request:

- Use the fetch function with the NYT API URL and your API key to make a request.

3. Check the Output:

- The console will display the JSON response or any errors.

4. Check the API Documentation

The NYT API documentation provides sample requests and responses. Use these examples to verify your tests:

- [NYT Article Search API Documentation](#)

5. Handle Errors

Test how the API behaves in error scenarios:

1. Invalid API Key:

- Use an incorrect API key and check the error message.

2. Invalid Query:

- Try an empty query (q=) or a malformed query.

3. Rate Limits:

- Make several requests quickly to see how the API handles rate limits.

By following these hints, you'll gain a clear understanding of how the NYT API works and ensure you're prepared to integrate it into your project. Let me know if you'd like help with any specific part!

Here are the [course contents](#) that may be relevant to your question.

4/28/2025, 12:52:50 PM

thumb_off_alt

etng@ucdavis.edu

so I am just testing if my API key is valid, and if it returns the names correct?

4/28/2025, 12:54:54 PM



CourseAssist AI

Yes, exactly! When testing your API key, you want to ensure that:

1. **The API key is valid:** The request should succeed without errors.
2. **The API returns the expected data:** You should verify that the response contains the correct structure and the fields you need, such as article titles, publication dates, and URLs.

Here's how you can approach this step-by-step:

Step-by-Step Testing Process

Step 1: Set Up the Request

- Use the NY Times API endpoint, such as the Article Search API:

plaintext

COPY

`https://api.nytimes.com/svc/search/v2/articlesearch.json?q=keyword&api-key=YOUR_`

- Replace `YOUR_API_KEY` with your actual API key.
- Replace `keyword` with a test query, such as "Davis" or "Sacramento".

Step 2: Make the Request

You can test the request using one of the following methods:

1. Postman:

- Enter the URL above in Postman.
- Click "Send" and check the response.

2. Browser Developer Console:

- Open the console (F12 -> "Console").
- Write JavaScript code to make the request using `fetch()`.

3. Python Script:

- Use Python's `requests` library to send the request and print the response.

Step 3: Check the Response

- Look for the following in the JSON response:
 - **Status:** Ensure the status is "OK".
 - **Response Data:** Check the `response.docs` array for article information.
 - Verify fields like `headline.main`, `web_url`, `pub_date`, etc.

Step 4: Handle Errors

- Test with an invalid API key and observe the error message in the response.

Step 5: Verify Relevant Fields

- Ensure the articles returned match your query (`q=Davis`).
- Confirm the fields you need for your project are present:
 - `headline.main`: Article title.
 - `web_url`: Link to the article.
 - `pub_date`: Publication date.

What to Do Next

Once you confirm the API key is valid and the response contains the expected data, you can proceed to integrate the backend and frontend. If you encounter issues or unexpected responses, let me know—I can help troubleshoot!

4/28/2025, 12:55:13 PM

thumbn_off_valtoff_alt

etng@ucdavis.edu

So there was this question that have been answered in class now does that mean I could take what I did in hw1, and not use svelte? I can only use flask and build my own app.py?: For 1. I believe they will be doing that, unless you are specifying that it needs to be on a different port.

For 2. it depends if you are using the Svelte for this project, or if you'll switch later. The material in dist will be served by flask in production mode. In the dev mode that I just setup uses Vite + Svelte to serve. If you are trying to do something that will work in dev and prod, then you'd want to use Svelte and integrate your work into the App.Svelte.

For 3. I put a link to a list of Flask tutorials, but the goal of homework 2 isn't focused on the backend. Really all you need to do is have it serve the API key which it can already do. If you want to have it do more, then you would need to work with flask.

4/30/2025, 10:15:11 AM



Yes, based on the clarified instructions and class discussion, you **can use your Homework 1 frontend (HTML, CSS, and JavaScript)** without converting it to Svelte. You only need to ensure that your backend uses Flask to securely handle the New York Times API key and serve your static files.

Here's how you can approach this:

Option 1: Use Your Homework 1 Frontend

1. Reuse Your HTML, CSS, and JavaScript:

- Take the static files you created for Homework 1 (e.g., `index.html`, `style.css`, and any JavaScript files).
- Place them in the appropriate folder in your project.

2. Serve Static Files Using Flask:

- Use Flask to serve your static assets (HTML, CSS, and JavaScript). Look into Flask functions that can help with serving files.

3. Handle the API Key in Flask:

- Create a Flask route to serve the NY Times API key securely to the frontend.
- Alternatively, proxy the API requests through Flask to keep the API key hidden.

Option 2: Use Svelte (Optional)

If you want to use Svelte, you can integrate your Homework 1 frontend into the Svelte structure. This would involve:

- Converting your HTML into Svelte components.
- Using Vite for development and ensuring the production build works with Flask.

Key Requirements

Regardless of whether you use your Homework 1 frontend or switch to Svelte, you must:

1. Use Flask for the Backend:

- Flask is required for handling the API key securely and serving static files.

2. Pull Live Articles from the NYT API:

- Ensure your app fetches articles dynamically based on the API response.

3. Ensure Compatibility in Production Mode:

- If you use Svelte, make sure the production build (`frontend/dist`) is served correctly by Flask.

Steps to Build Your Flask App

If you're not using Svelte, here's how you can set up your Flask app:

1. Serve Static Files

Think about how Flask can serve your HTML, CSS, and JavaScript files. Research Flask routes and functions for serving files from specific folders.

2. Handle the API Key

Consider how you would create a secure route in Flask to provide the NY Times API key to the frontend.

3. Proxy API Requests (Optional)

If you want to keep the API key hidden, explore how Flask can act as a proxy for NYT API requests.

Final Notes

- **Frontend Choice:** You are not required to use Svelte for Homework 2. You can stick with your Homework 1 frontend and integrate it with Flask.
- **Focus on Flask:** The backend's primary purpose is to securely handle the API key and serve the frontend. Any additional backend functionality is optional.

If you need further clarification or help with specific steps, feel free to ask!

Here are the [course contents](#) that may be relevant to your question.

4/30/2025, 10:15:33 AM

thumb_off_valtoff_alt

etng@ucdavis.edu

So based on the HW2 description, would I need to use the starter code, if I only wanted to use Flask

4/30/2025, 10:19:24 AM



No, you are **not required to use the starter code** if you only want to use Flask for Homework 2. The starter code is provided to help you set up a project that uses **Svelte** for the frontend and Flask for the backend. However, if you prefer to use your own frontend (e.g., your Homework 1 code) and only use Flask for the backend, that is perfectly acceptable, as long as you meet the following requirements:

Key Requirements for HW2

1. Use Flask to Handle the API Key:

- You must use Flask to securely handle the New York Times API key. This means the API key should not be hardcoded in the frontend.
- Think about creating a Flask route or using a proxy to ensure the API key remains hidden.

2. Serve Static Files:

- Flask should serve your static files (HTML, CSS, and JavaScript) to the browser. Organize your project structure to make this straightforward.

3. Fetch Live Articles:

- Your app must dynamically fetch articles from the NY Times API and display them in the frontend. Consider how the frontend will communicate with the backend to retrieve data.

4. Ensure Compatibility:

- Your submission must work in production mode (i.e., when Flask serves the static files and handles API requests).

General Guidance

If you don't want to use the starter code, here are some things to think about:

1. Setting Up Your Flask Backend

- Plan how to define routes to serve static files and securely handle the API key.

2. Organizing Your Project Folder

- Decide on a folder structure to separate static files and backend code for clarity.

3. Testing Your Flask App

- Test locally to ensure Flask is correctly serving files and handling API requests.

4. Submitting Your Project

- Ensure your submission includes all necessary files and works when deployed.

Pros and Cons of Using Starter Code

Pros:

- The starter code is pre-configured to work with Svelte and Flask.
- It provides a clear structure for development and production modes.

Cons:

- If you don't want to use Svelte, you may find the starter code unnecessary and prefer to set up your own Flask app.

Final Notes

- You are free to use your own frontend and Flask setup as long as you meet the requirements outlined in the homework description.
- If you decide not to use the starter code, focus on ensuring your Flask app is properly configured to serve static files and handle the API key securely.

Let me know if you need help with any specific part of the setup!

Here are the [course contents](#) that may be relevant to your question.

4/30/2025, 10:19:42 AM

thumb_off_alt

etng@ucdavis.edu

Therefore, using flask for hw2, without the starter code will fulfill all the requirements for the homework? Tell me all the required things I need to succeed on this homework and complete all the requirements

4/30/2025, 10:22:46 AM



CourseAssist AI

Yes, using Flask without the starter code can fulfill all the requirements for Homework 2, as long as you meet the specific criteria outlined in the homework description. Below is a list of helpful hints to guide you in completing Homework 2:

Hints for HW2 Requirements

1. Use Flask for the Backend

- Flask is required to securely handle the New York Times API key and serve static files.
 - Think about setting up routes in Flask to handle your backend logic and serve your frontend files.
-

2. Handle the NYT API Key Securely

- The API key should not be hardcoded in the frontend for security reasons.
 - Consider storing the API key in a secure place (e.g., environment variable) and creating a way for the backend to serve or proxy API requests.
-

3. Serve Static Files

- Your frontend files (HTML, CSS, JavaScript) need to be served by Flask.
 - Research how Flask can serve static files and organize your project folder accordingly.
-

4. Fetch Live Articles from the NYT API

- The app must dynamically fetch articles based on the NYT API response.
 - Think about how to use Flask to either proxy API requests or serve the API key securely to the frontend.
-

5. Display Articles in the Frontend

- The frontend must dynamically show articles based on the NYT API response.
 - Use JavaScript to fetch data and update the DOM. Consider how you want to display article details like title, URL, and publication date.
-

6. Test Your Application

- Test locally by running your Flask app and accessing it via `http://localhost:8000`.
 - Verify that static files are served correctly and that articles are displayed dynamically in the frontend.
-

7. Follow Submission Guidelines

- Ensure your project folder is well-organized and includes all necessary files for grading.
- Include the required Docker configuration files to allow graders to build and test your app.

Checklist for Success

1. Flask backend is set up and running.
2. Static files (HTML, CSS, JavaScript) are served correctly.
3. API key is securely handled (via a route or proxy).
4. Articles are dynamically fetched and displayed in the frontend.
5. Layout matches design requirements.
6. Project folder is well-organized and includes all necessary files.
7. Submission works in production mode.

By following these hints and meeting the requirements, you can successfully complete Homework 2 without using the starter code. Let me know if you need guidance on any specific part of the process!

Here are the [course contents](#) that may be relevant to your question.

4/30/2025, 10:23:10 AM

thumb_up_off_alt

etng@ucdavis.edu

There seems to be tasks called production mode and unit testing. Is production mode required? How would I do unit testing?

4/30/2025, 10:42:00 AM



CourseAssist AI

Production Mode

Yes, production mode is required because the graders will test your submission in production mode. This means your Flask backend should serve the static files and handle API requests in a way that works without relying on development tools like Vite (used in the starter code).

How to Ensure Production Mode Works

1. Serve Static Files with Flask:

- Research how to use Flask to serve your `index.html`, CSS, and JavaScript files. Consider placing these files in a folder like `static` or `templates`, depending on your setup.

Hint: Look into Flask's `send_from_directory` function for serving files.

2. Handle the NYT API Key Securely:

- Explore ways to use Flask to proxy API requests or securely serve the API key to ensure it is not exposed in your frontend code.

3. Test Locally in Production Mode:

- Run your Flask app locally and verify that it works as expected when accessed via `http://localhost:8000`.

4. Docker Configuration:

- Review how to set up your `docker-compose.yml` file to ensure your Flask app runs in production mode.

Unit Testing

Unit testing is required for Homework 2, as 25% of the grade is allocated to unit testing and repository management. Unit tests ensure that specific parts of your code work as intended.

How to Do Unit Testing

Unit testing applies to both the backend (Flask) and frontend (JavaScript). Here's how you can approach it:

1. Backend Unit Testing (Flask)

1. Set Up a Testing Framework:

- Research Python's built-in `unittest` module or third-party libraries like `pytest`.

2. Write Tests for Flask Routes:

- Think about how to test your Flask routes to ensure they return the correct responses (e.g., serving the API key or proxying requests).

Hint: Look into Flask's `test_client()` for testing routes.

3. Mock External API Calls:

- Investigate libraries like `unittest.mock` to simulate calls to the NYT API so your tests don't depend on live API responses.
-

2. Frontend Unit Testing (JavaScript)

1. Set Up a JavaScript Testing Framework:

- Explore frameworks like Jest or Mocha for testing your JavaScript code.

2. Test Data Fetching:

- Focus on verifying that your JavaScript correctly fetches data from the backend and handles responses properly.

Hint: Look into mocking functions like `fetch` to simulate backend responses.

3. Mock Backend Responses:

- Research tools for simulating responses from your Flask backend to test how your frontend handles them.
-

3. Repository Management

Good repository management is also part of the unit testing grade. Ensure the following:

1. Version Control:

- Use Git for version control. Commit regularly with meaningful commit messages.

2. Branching:

- Use feature branches for different parts of your project (e.g., one branch for backend, another for frontend).

3. Organization:

- Keep your repository well-organized, with clear separation between frontend and backend code.

4. Documentation:

- Include a `README.md` file explaining how to run your app and tests.
-

Checklist for Success

1. Production Mode:

- Ensure Flask serves static files correctly.
- Verify the app works when accessed via `http://localhost:8000`.

2. Unit Testing:

- Write tests for Flask routes.
- Write tests for frontend JavaScript logic.
- Mock external API calls to isolate functionality.

3. Repository Management:

- Use Git effectively.
- Organize your project structure.
- Include clear documentation.

By ensuring production mode works and writing unit tests for both the backend and frontend, you can fulfill all the requirements for Homework 2. Let me know if you need help with specific parts of production mode or unit testing!

Here are the [course contents](#) that may be relevant to your question.

4/30/2025, 10:42:37 AM

thumb_off_alt